01 LC 55 | Jump Game (Medium)
Problem: Starting at index 0, nums[i] is the maximum forward jump length from i. Return whether the last index can be reached.
Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 10^5

```
class Solution {
public:
  bool canJump(vector<int>& nums) {
    int mx = 0;
    for (int i = 0; i < nums.size(); ++i) {
      if (mx < i) {
        return false;
      }
      mx = max(mx, i + nums[i]);
    }
    return true;
  }
};
```

40 LC 2366 | Minimum Replacements to Sort the Array (Hard)
Problem: You are given a 0-indexed integer array nums. In one operation you can replace any element of the array with any two elements that sum to it. - For example, consider nums = [5,6,7]. In one operation, we can replace nums[1] with 2 and 4 and convert nums to [5,2,4,7]. Return the minimum number of operations to make an array that is sorted in non-decreasing order.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^9

```
class Solution {
public:
  long long minimumReplacement(vector<int>& nums)
    {
    long long ans = 0;
    int n = nums.size();
    int mx = nums[n - 1];
    for (int i = n - 2; i >= 0; --i) {
      if (nums[i] <= mx) {
        mx = nums[i];
        continue;
      }
      int k = (nums[i] + mx - 1) / mx;
      ans += k - 1;
      mx = nums[i] / k;
    }
    return ans;
  }
};
```

43 LC 2491 | Divide Players Into Teams of Equal Skill (Medium)
Problem: Pair all players so every team's skill sum is equal. Team chemistry is the product of its players' skills. Return total chemistry, or -1 if no valid pairing exists.
Constraints: 2 <= skill.length <= 10^5; skill.length is even.; 1 <= skill[i] <= 1000

```
class Solution {
public:
  long long dividePlayers(vector<int>& skill) {
    int teams = skill.size() / 2;
    int sum = accumulate(skill.begin(),
      skill.end(), 0);
    if (sum % teams) return -1;
    int target = sum / teams;
    array<int, 1001> waiting{};
    long long chemistry = 0;
    int matched = 0;
    for (int x : skill) {
      int y = target - x;
      if (y < 1 || y > 1000) return -1;
      if (waiting[y]) { --waiting[y]; ++matched;
        chemistry += 1LL * x * y; }
      else ++waiting[x];
    }
    return matched == teams ? chemistry : -1;
  }
};
```

02 LC 45 | Jump Game II (Medium)
Problem: Starting at index 0, jump forward by at most nums[i] positions from index i. Return the minimum jumps needed to reach the last index, which is guaranteed reachable.
Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 1000; It's guaranteed that you can reach nums[n - 1].

```
class Solution {
public:
  int jump(vector<int>& nums) {
    int ans = 0, mx = 0, last = 0;
    for (int i = 0; i < nums.size() - 1; ++i) {
      mx = max(mx, i + nums[i]);
      if (last == i) {
        ++ans;
        last = mx;
      }
    }
    return ans;
  }
};
```

06 LC 56 | Merge Intervals (Medium)
Problem: Merge all overlapping closed intervals and return nonoverlapping intervals covering exactly the same points.
Constraints: 1 <= intervals.length <= 10^4; intervals[i].length == 2; 0 <= start_i <= end_i <= 10^4

```
class Solution {
public:
  vector<vector<int>> merge(vector<vector<int>>&
    intervals) {
    ranges::sort(intervals);
    int st = intervals[0][0], ed =
      intervals[0][1];
    vector<vector<int>> ans;
    for (int i = 1; i < intervals.size(); ++i) {
      if (ed < intervals[i][0]) {
        ans.push_back({st, ed});
        st = intervals[i][0];
        ed = intervals[i][1];
      } else {
        ed = max(ed, intervals[i][1]);
      }
    }
    ans.push_back({st, ed});
    return ans;
  }
};
```

09 LC 763 | Partition Labels (Medium)
Problem: Split s into as many contiguous parts as possible so each letter appears in only one part. Return the lengths of the parts in order.
Constraints: 1 <= s.length <= 500; s consists of lowercase English letters.

```
class Solution {
public:
  vector<int> partitionLabels(string s) {
    int last[26] = {0};
    int n = s.size();
    for (int i = 0; i < n; ++i) {
      last[s[i] - 'a'] = i;
    }
    vector<int> ans;
    int mx = 0, j = 0;
    for (int i = 0; i < n; ++i) {
      mx = max(mx, last[s[i] - 'a']);
      if (mx == i) {
        ans.push_back(i - j + 1);
        j = i + 1;
      }
    }
    return ans;
  }
};
```

03 LC 121 | Best Time to Buy and Sell Stock (Easy)
Problem: Choose one day to buy and a later day to sell one stock. Return the maximum profit, or 0 when no profitable transaction exists.
Constraints: 1 <= prices.length <= 10^5; 0 <= prices[i] <= 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int ans = 0, mi = prices[0];
    for (int& v : prices) {
      ans = max(ans, v - mi);
      mi = min(mi, v);
    }
    return ans;
  }
};
```

07 LC 435 | Non-overlapping Intervals (Medium)
Problem: Given an array of intervals intervals where intervals[i] = [start_i, end_i], return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping. Note that intervals which only touch at a point are non-overlapping. For example, [1, 2] and [2, 3] are non-overlapping.
Constraints: 1 <= intervals.length <= 10^5; intervals[i].length == 2; -5 * 10^4 <= start_i < end_i <= 5 * 10^4

```
class Solution {
public:
  int eraseOverlapIntervals(vector<vector<int>>&
    intervals) {
    ranges::sort(intervals, [](const vector<int>&
      a, const vector<int>& b) {
      return a[1] < b[1];
    });
    int ans = intervals.size();
    int pre = INT_MIN;
    for (const auto& e : intervals) {
      int l = e[0], r = e[1];
      if (pre <= l) {
        --ans;
        pre = r;
      }
    }
    return ans;
  }
};
```

08 LC 452 | Minimum Number of Arrows to Burst Balloons (Medium)
Problem: Each closed interval is a balloon's horizontal extent. A vertical arrow at x bursts all intervals containing x. Return the minimum arrows needed to burst every balloon; arrows are unlimited and may be fired at any real x.
Constraints: 1 <= points.length <= 10^5; points[i].length == 2; -2^31 <= x_start < x_end <= 2^31 - 1

```
class Solution {
public:
  int findMinArrowShots(vector<vector<int>>&
    points) {
    ranges::sort(points, [](vector<int>& a,
      vector<int>& b) {
      return a[1] < b[1];
    });
    int ans = 0;
    long long last = -(1LL << 60);
    for (auto& p : points) {
      int a = p[0], b = p[1];
      if (a > last) {
        ++ans;
        last = b;
      }
    }
    return ans;
  }
};
```

04 LC 122 | Best Time to Buy and Sell Stock II (Medium)
Problem: Maximize stock profit with any number of buy/sell transactions, holding at most one share at a time. Buying and selling on the same day is allowed; return the maximum profit.
Constraints: 1 <= prices.length <= 3 * 10^4; 0 <= prices[i] <= 10^4

```
class Solution {
public:
  int maxProfit(vector<int>& prices) {
    int ans = 0;
    for (int i = 1; i < prices.size(); ++i) ans +=
      max(0, prices[i] - prices[i - 1]);
    return ans;
  }
};
```

10 LC 53 | Maximum Subarray (Medium)
Problem: Return the largest sum of a nonempty contiguous subarray of nums.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int maxSubArray(vector<int>& nums) {
    int ans = nums[0], f = nums[0];
    for (int i = 1; i < nums.size(); ++i) {
      f = max(f, 0) + nums[i];
      ans = max(ans, f);
    }
    return ans;
  }
};
```

37 LC 2160 | Minimum Sum of Four Digit Number After Splitting Digits (Easy)
Problem: Split the four decimal digits of num between two new nonnegative integers, using every digit once. Leading zeros are allowed. Return the minimum possible sum.
Constraints: 1000 <= num <= 9999

```
class Solution {
public:
  int minimumSum(int num) {
    vector<int> nums;
    while (num) {
      nums.push_back(num % 10);
      num /= 10;
    }
    ranges::sort(nums);
    return 10 * (nums[0] + nums[1]) + nums[2] +
      nums[3];
  }
};
```

39 LC 2335 | Minimum Amount of Time to Fill Cups (Easy)
Problem: Each second fill either two cups of different types or one cup of any type. Return the minimum seconds needed to fill the three required type counts.
Constraints: amount.length == 3; 0 <= amount[i] <= 100

```
class Solution {
public:
  int fillCups(vector<int>& amount) {
    ranges::sort(amount);
    if (amount[0] + amount[1] <= amount[2]) {
      return amount[2];
    }
    return (amount[0] + amount[1] + amount[2] + 1)
      / 2;
  }
};
```

05 LC 134 | Gas Station (Medium)

Problem: Stations form a circular route. Collect gas[i] at station i and spend cost[i] to reach the next station. Starting with an empty, unlimited tank, return a starting index that completes the circuit, or -1. A feasible start is unique.
Constraints: n == gas.length == cost.length; 1 <= n <= 10^5; 0 <= gas[i], cost[i] <= 10^4; The input is generated such that the answer is unique.

```
class Solution {
public:
  int canCompleteCircuit(vector<int>& gas,
    vector<int>& cost) {
    int n = gas.size();
    int i = n - 1, j = n - 1;
    int cnt = 0, s = 0;
    while (cnt < n) {
      s += gas[j] - cost[j];
      ++cnt;
      j = (j + 1) % n;
      while (s < 0 && cnt < n) {
        --i;
        s += gas[i] - cost[i];
        ++cnt;
      }
    }
    return s < 0 ? -1 : i;
  }
};
```

36 LC 2038 | Remove Colored Pieces if Both Neighbors are the Same Color (Medium)

Problem: Alice removes 'A' pieces and Bob removes 'B' pieces, with Alice first. A piece can be removed only when both neighbors have the same color as it; endpoints cannot be removed. A player with no legal move loses. Return whether Alice wins optimally.
Constraints: 1 <= colors.length <= 10^5; colors consists of only the letters 'A' and 'B'

```
class Solution {
public:
  bool winnerOfGame(string colors) {
    int n = colors.size();
    int a = 0, b = 0;
    for (int i = 0, j = 0; i < n; i = j) {
      while (j < n && colors[j] == colors[i]) {
        ++j;
      }
      int m = j - i - 2;
      if (m > 0) {
        if (colors[i] == 'A') {
          a += m;
        } else {
          b += m;
        }
      }
    }
    return a > b;
  }
};
```

11 LC 406 | Queue Reconstruction by Height (Medium)

Problem: Each person [h,k] has height h and exactly k people of height at least h standing before them. Reconstruct a queue satisfying every pair and return it; a valid reconstruction exists.
Constraints: 1 <= people.length <= 2000; 0 <= h_i <= 10^6; 0 <= k_i < people.length; It is guaranteed that the queue can be reconstructed.

```
class Solution {
public:
  vector<vector<int>>
    reconstructQueue(vector<vector<int>>& people)
    {
    ranges::sort(people, [](const auto& a, const
      auto& b) {
      return a[0] != b[0] ? a[0] < b[0] : a[1] >
        b[1];
    });
    int n = people.size();
    vector<int> tree(n + 1);
    for (int i = 1; i <= n; ++i) tree[i] = i & -i;
    vector<vector<int>> result(n);
    for (const auto& person : people) {
      int rank = person[1] + 1, index = 0;
      for (unsigned step =
        bit_floor(static_cast<unsigned>(n)); step;
        step >>= 1) {
        int candidate = index +
          static_cast<int>(step);
        if (candidate <= n && tree[candidate] <
          rank) {
          index = candidate; rank -=
            tree[candidate];
        }
      }
      result[index] = person;
      for (int i = index + 1; i <= n; i += i & -i)
        --tree[i];
    }
    return result;
  }
};
```

25 LC 659 | Split Array into Consecutive Subsequences (Medium)

Problem: Partition sorted nums into one or more subsequences, each of consecutive integers and length at least three. Use every occurrence exactly once. Return whether such a partition exists.
Constraints: 1 <= nums.length <= 10^4; -1000 <= nums[i] <= 1000; nums is sorted in non-decreasing order.

```
class Solution {
public:
  bool isPossible(vector<int>& nums) {
    unordered_map<int, priority_queue<int,
      vector<int>, greater<int>>> d;
    for (int v : nums) {
      if (d.count(v - 1)) {
        auto& q = d[v - 1];
        d[v].push(q.top() + 1);
        q.pop();
        if (q.empty()) {
          d.erase(v - 1);
        }
      } else {
        d[v].push(1);
      }
    }
    for (auto& [_, v] : d) {
      if (v.top() < 3) {
        return false;
      }
    }
    return true;
  }
};
```

12 LC 135 | Candy (Hard)
Problem: There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings. You are giving candies to these children subjected to the following requirements: - Each child must have at least one candy. - Children with a higher rating get more candies than their neighbors. Return the minimum number of candies you need to have to distribute the candies to the children.
Constraints: 1 <= n == ratings.length <= 5 * 10^4; 0 <= ratings[i] <= 5 * 10^4

```
class Solution {
public:
  int candy(vector<int>& ratings) {
    int n = ratings.size();
    vector<int> left(n, 1);
    vector<int> right(n, 1);
    for (int i = 1; i < n; ++i) {
      if (ratings[i] > ratings[i - 1]) {
        left[i] = left[i - 1] + 1;
      }
    }
    for (int i = n - 2; ~i; --i) {
      if (ratings[i] > ratings[i + 1]) {
        right[i] = right[i + 1] + 1;
      }
    }
    int ans = 0;
    for (int i = 0; i < n; ++i) {
      ans += max(left[i], right[i]);
    }
    return ans;
  }
};
```

26 LC 767 | Reorganize String (Medium)
Problem: Given a string s, rearrange the characters of s so that any two adjacent characters are not the same. Return any possible rearrangement of s or return "" if not possible.
Constraints: 1 <= s.length <= 500; s consists of lowercase English letters.

```
class Solution {
public:
  string reorganizeString(string s) {
    vector<int> cnt(26);
    for (char& c : s) ++cnt[c - 'a'];
    int mx = *max_element(cnt.begin(), cnt.end());
    int n = s.size();
    if (mx > (n + 1) / 2) return "";
    vector<vector<int>> m;
    for (int i = 0; i < 26; ++i) {
      if (cnt[i]) m.push_back({cnt[i], i});
    }
    ranges::sort(m);
    ranges::reverse(m);
    string ans = s;
    int k = 0;
    for (auto& e : m) {
      int v = e[0], i = e[1];
      while (v--) {
        ans[k] = 'a' + i;
        k += 2;
        if (k >= n) k = 1;
      }
    }
    return ans;
  }
};
```

13 LC 455 | Assign Cookies (Easy)
Problem: Each child needs a cookie of size at least its greed factor. Give at most one cookie to each child and use each cookie once. Return the maximum number of satisfied children.
Constraints: 1 <= g.length <= 3 * 10^4; 0 <= s.length <= 3 * 10^4; 1 <= g[i], s[j] <= 2^31 - 1

```
class Solution {
public:
  int findContentChildren(vector<int>& g,
    vector<int>& s) {
    ranges::sort(g);
    ranges::sort(s);
    int m = g.size(), n = s.size();
    for (int i = 0, j = 0; i < m; ++i) {
      while (j < n && s[j] < g[i]) {
        ++j;
      }
      if (j++ >= n) {
        return i;
      }
    }
    return m;
  }
};
```

44 LC 2578 | Split With Minimum Sum (Easy)
Problem: Distribute all decimal digits of num between two integers, allowing leading zeros and arbitrary digit order. Return their smallest possible sum.
Constraints: 10 <= num <= 10^9

```
class Solution {
public:
  int splitNum(int num) {
    int cnt[10]{};
    int n = 0;
    for (; num; num /= 10) {
      ++cnt[num % 10];
      ++n;
    }
    int ans[2]{};
    for (int i = 0, j = 0; i < n; ++i) {
      while (cnt[j] == 0) {
        ++j;
      }
      --cnt[j];
      ans[i & 1] = ans[i & 1] * 10 + j;
    }
    return ans[0] + ans[1];
  }
};
```

47 LC 3075 | Maximize Happiness of Selected Children (Medium)
Problem: Select exactly k children sequentially. Each selection adds that child's current happiness, then decreases every unselected child's happiness by 1, never below 0. Return the maximum total happiness.
Constraints: 1 <= n == happiness.length <= 2 * 10^5; 1 <= happiness[i] <= 10^8; 1 <= k <= n

```
class Solution {
public:
  long long maximumHappinessSum(vector<int>&
    happiness, int k) {
    sort(happiness.rbegin(), happiness.rend());
    long long ans = 0;
    for (int i = 0, n = happiness.size(); i < k;
      ++i) {
      int x = happiness[i] - i;
      ans += max(x, 0);
    }
    return ans;
  }
};
```

14 LC 860 | Lemonade Change (Easy)

Problem: Lemonade costs $5. Customers pay with $5, $10, or $20 in the supplied order. Starting with no cash, return whether exact change can be given to every customer immediately.
Constraints: 1 <= bills.length <= 10^5; bills[i] is either 5, 10, or 20.

```
class Solution {
public:
  bool lemonadeChange(vector<int>& bills) {
    int five = 0, ten = 0;
    for (int v : bills) {
      if (v == 5) {
        ++five;
      } else if (v == 10) {
        ++ten;
        --five;
      } else {
        if (ten) {
          --ten;
          --five;
        } else {
          five -= 3;
        }
      }
      if (five < 0) {
        return false;
      }
    }
    return true;
  }
};
```

38 LC 2279 | Maximum Bags With Full Capacity of Rocks (Medium)

Problem: You have n bags numbered from 0 to n - 1. You are given two 0-indexed integer arrays capacity and rocks. The ith bag can hold a maximum of capacity[i] rocks and currently contains rocks[i] rocks. You are also given an integer additionalRocks, the number of additional rocks you can place in any of the bags. Return the maximum number of bags that could have full capacity after placing the additional rocks in some bags.
Constraints: n == capacity.length == rocks.length; 1 <= n <= 5 * 10^4; 1 <= capacity[i] <= 10^9; 0 <= rocks[i] <= capacity[i]; 1 <= additionalRocks <= 10^9

```
class Solution {
public:
  int maximumBags(vector<int>& capacity,
    vector<int>& rocks, int additionalRocks) {
    int n = rocks.size();
    for (int i = 0; i < n; ++i) {
      capacity[i] -= rocks[i];
    }
    ranges::sort(capacity);
    for (int i = 0; i < n; ++i) {
      additionalRocks -= capacity[i];
      if (additionalRocks < 0) {
        return i;
      }
    }
    return n;
  }
};
```

15 LC 605 | Can Place Flowers (Easy)

Problem: You have a long flowerbed in which some of the plots are planted, and some are not. However, flowers cannot be planted in adjacent plots. Given an integer array flowerbed containing 0's and 1's, where 0 means empty and 1 means not empty, and an integer n, return true if n new flowers can be planted in the flowerbed without violating the no-adjacent-flowers rule and false otherwise.
Constraints: 1 <= flowerbed.length <= 2 * 10^4; flowerbed[i] is 0 or 1.; There are no two adjacent flowers in flowerbed.; 0 <= n <= flowerbed.length

```
class Solution {
public:
  bool canPlaceFlowers(vector<int>& flowerbed, int
    n) {
    int m = flowerbed.size();
    for (int i = 0; i < m; ++i) {
      int l = i == 0 ? 0 : flowerbed[i - 1];
      int r = i == m - 1 ? 0 : flowerbed[i + 1];
      if (l + flowerbed[i] + r == 0) {
        flowerbed[i] = 1;
        --n;
      }
    }
    return n <= 0;
  }
};
```

17 LC 376 | Wiggle Subsequence (Medium)

Problem: Return the length of a longest subsequence whose consecutive differences are nonzero and strictly alternate in sign. The subsequence need not be contiguous; zero- or one-difference sequences are valid wiggles.
Constraints: 1 <= nums.length <= 1000; 0 <= nums[i] <= 1000

```
class Solution {
public:
  int wiggleMaxLength(vector<int>& nums) {
    int n = nums.size();
    int ans = 1;
    vector<int> f(n, 1);
    vector<int> g(n, 1);
    for (int i = 1; i < n; ++i) {
      for (int j = 0; j < i; ++j) {
        if (nums[j] < nums[i]) {
          f[i] = max(f[i], g[j] + 1);
        } else if (nums[j] > nums[i]) {
          g[i] = max(g[i], f[j] + 1);
        }
      }
      ans = max({ans, f[i], g[i]});
    }
    return ans;
  }
};
```

28 LC 921 | Minimum Add to Make Parentheses Valid (Medium)

Problem: Return the minimum number of '(' or ')' insertions needed to make s balanced. Insertions may be made at any positions.
Constraints: 1 <= s.length <= 1000; s[i] is either '(' or ')'.

```
class Solution {
public:
  int minAddToMakeValid(string s) {
    int ans = 0, cnt = 0;
    for (char c : s) {
      if (c == '(')
        ++cnt;
      else if (cnt)
        --cnt;
      else
        ++ans;
    }
    ans += cnt;
    return ans;
  }
};
```

16 LC 665 | Non-decreasing Array (Medium)

Problem: Given an array nums with n integers, your task is to check if it could become non-decreasing by modifying at most one element. We define an array is non-decreasing if nums[i] <= nums[i + 1] holds for every i (0-based) such that (0 <= i <= n - 2).
Constraints: n == nums.length; 1 <= n <= 10^4; -10^5 <= nums[i] <= 10^5

```
class Solution {
public:
  bool checkPossibility(vector<int>& nums) {
    int n = nums.size();
    for (int i = 0; i < n - 1; ++i) {
      int a = nums[i], b = nums[i + 1];
      if (a > b) {
        nums[i] = b;
        if (is_sorted(nums.begin(), nums.end())) {
          return true;
        }
        nums[i] = a;
        nums[i + 1] = a;
        return is_sorted(nums.begin(),
          nums.end());
      }
    }
    return true;
  }
};
```

19 LC 621 | Task Scheduler (Medium)

Problem: You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label. Return the minimum number of CPU intervals required to complete all tasks.
Constraints: 1 <= tasks.length <= 10^4; tasks[i] is an uppercase English letter.; 0 <= n <= 100

```
class Solution {
public:
  int leastInterval(vector<char>& tasks, int n) {
    vector<int> cnt(26);
    int x = 0;
    for (char c : tasks) {
      c -= 'A';
      ++cnt[c];
      x = max(x, cnt[c]);
    }
    int s = 0;
    for (int v : cnt) {
      s += v == x;
    }
    return max((int) tasks.size(), (x - 1) * (n +
      1) + s);
  }
};
```

33 LC 1833 | Maximum Ice Cream Bars (Medium)

Problem: Each ice cream bar costs costs[i] coins and may be bought once. Given coins, return the maximum number of bars that can be purchased, in any order.
Constraints: costs.length == n; 1 <= n <= 10^5; 1 <= costs[i] <= 10^5; 1 <= coins <= 10^8

```
class Solution {
public:
  int maxIceCream(vector<int>& costs, int coins) {
    ranges::sort(costs);
    int n = costs.size();
    for (int i = 0; i < n; ++i) {
      if (coins < costs[i]) return i;
      coins -= costs[i];
    }
    return n;
  }
};
```

18 LC 738 | Monotone Increasing Digits (Medium)

Problem: An integer has monotone increasing digits if and only if each pair of adjacent digits x and y satisfy x <= y. Given an integer n, return the largest number that is less than or equal to n with monotone increasing digits.
Constraints: 0 <= n <= 10^9

```
class Solution {
public:
  int monotoneIncreasingDigits(int n) {
    string s = to_string(n);
    int i = 1;
    for (; i < s.size() && s[i - 1] <= s[i]; ++i)
      ;
    if (i < s.size()) {
      for (; i > 0 && s[i - 1] > s[i]; --i) {
        --s[i - 1];
      }
      ++i;
      for (; i < s.size(); ++i) {
        s[i] = '9';
      }
    }
    return stoi(s);
  }
};
```

32 LC 1710 | Maximum Units on a Truck (Easy)

Problem: Each box type has a count and units per box. Load at most truckSize boxes of any types and return the maximum total units.
Constraints: 1 <= boxTypes.length <= 1000; 1 <= numberOfBoxes_i, numberOfUnitsPerBox_i <= 1000; 1 <= truckSize <= 10^6

```
class Solution {
public:
  int maximumUnits(vector<vector<int>>& boxTypes,
    int truckSize) {
    int cnt[1001] = {0};
    for (auto& e : boxTypes) {
      int a = e[0], b = e[1];
      cnt[b] += a;
    }
    int ans = 0;
    for (int b = 1000; b > 0 && truckSize > 0;
      --b) {
      int a = cnt[b];
      if (a) {
        ans += b * min(truckSize, a);
        truckSize -= a;
      }
    }
    return ans;
  }
};
```

45 LC 2587 | Rearrange Array to Maximize Prefix Score (Medium)

Problem: Reorder nums to maximize the number of strictly positive prefix sums. Return that maximum count; each input occurrence must be used exactly once.
Constraints: 1 <= nums.length <= 10^5; -10^6 <= nums[i] <= 10^6

```
class Solution {
public:
  int maxScore(vector<int>& nums) {
    sort(nums.rbegin(), nums.rend());
    long long s = 0;
    int n = nums.size();
    for (int i = 0; i < n; ++i) {
      s += nums[i];
      if (s <= 0) {
        return i;
      }
    }
    return n;
  }
};
```

20 LC 1029 | Two City Scheduling (Medium)
Problem: A company is planning to interview 2n people. Given the array costs where costs[i] = [aCost_i, bCost_i], the cost of flying the ith person to city a is aCost_i, and the cost of flying the ith person to city b is bCost_i. Return the minimum cost to fly every person to a city such that exactly n people arrive in each city.
Constraints: 2 * n == costs.length; 2 <= costs.length <= 100; costs.length is even.; 1 <= aCost_i, bCost_i <= 1000

```
class Solution {
public:
  int twoCitySchedCost(vector<vector<int>>& costs)
    {
    ranges::sort(costs, [](const vector<int>& a,
      const vector<int>& b) {
      return a[0] - a[1] < b[0] - b[1];
    });
    int n = costs.size() / 2;
    int ans = 0;
    for (int i = 0; i < n; ++i) {
      ans += costs[i][0] + costs[i + n][1];
    }
    return ans;
  }
};
```

21 LC 646 | Maximum Length of Pair Chain (Medium)
Problem: You are given an array of n pairs pairs where pairs[i] = [left_i, right_i] and left_i < right_i. A pair p2 = [c, d] follows a pair p1 = [a, b] if b < c. A chain of pairs can be formed in this fashion. Return the length longest chain which can be formed. You do not need to use up all the given intervals. You can select pairs in any order.
Constraints: n == pairs.length; 1 <= n <= 1000; -1000 <= left_i < right_i <= 1000

```
class Solution {
public:
  int findLongestChain(vector<vector<int>>& pairs)
    {
    ranges::sort(pairs, {}, [](const auto& p) {
      return p[1]; });
    int ans = 0, pre = INT_MIN;
    for (const auto& p : pairs) {
      if (pre < p[0]) {
        pre = p[1];
        ++ans;
      }
    }
    return ans;
  }
};
```

41 LC 2405 | Optimal Partition of String (Medium)
Problem: Given a string s, partition the string into one or more substrings such that the characters in each substring are unique. That is, no letter appears in a single substring more than once. Return the minimum number of substrings in such a partition. Note that each character should belong to exactly one substring in a partition.
Constraints: 1 <= s.length <= 10^5; s consists of only English lowercase letters.

```
class Solution {
public:
  int partitionString(string s) {
    int ans = 1, mask = 0;
    for (char& c : s) {
      int x = c - 'a';
      if (mask >> x & 1) {
        ++ans;
        mask = 0;
      }
      mask |= 1 << x;
    }
    return ans;
  }
};
```

22 LC 881 | Boats to Save People (Medium)
Problem: You are given an array people where people[i] is the weight of the ith person, and an infinite number of boats where each boat can carry a maximum weight of limit. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most limit. Return the minimum number of boats to carry every given person.
Constraints: 1 <= people.length <= 5 * 10^4; 1 <= people[i] <= limit <= 3 * 10^4

```
class Solution {
public:
  int numRescueBoats(vector<int>& people, int
    limit) {
    ranges::sort(people);
    int ans = 0;
    for (int i = 0, j = people.size() - 1; i <= j;
      --j) {
      if (people[i] + people[j] <= limit) {
        ++i;
      }
      ++ans;
    }
    return ans;
  }
};
```

35 LC 1963 | Minimum Number of Swaps to Make the String Balanced (Medium)
Problem: s has equal counts of '[' and ']'. One swap exchanges any two positions. Return the minimum swaps needed to make the bracket string balanced.
Constraints: n == s.length; 2 <= n <= 10^6; n is even.; s[i] is either '[' or ']'.; The number of opening brackets '[' equals n / 2, and the number of closing brackets ']' equals n / 2.

```
class Solution {
public:
  int minSwaps(string s) {
    int x = 0;
    for (char& c : s) {
      if (c == '[') {
        ++x;
      } else if (x) {
        --x;
      }
    }
    return (x + 1) / 2;
  }
};
```

42 LC 2410 | Maximum Matching of Players With Trainers (Medium)
Problem: Match players to trainers when trainer capacity is at least player ability. Each participant is used at most once. Return the maximum number of matches.
Constraints: 1 <= players.length, trainers.length <= 10^5; 1 <= players[i], trainers[j] <= 10^9

```
class Solution {
public:
  int matchPlayersAndTrainers(vector<int>&
    players, vector<int>& trainers) {
    ranges::sort(players);
    ranges::sort(trainers);
    int m = players.size(), n = trainers.size();
    for (int i = 0, j = 0; i < m; ++i, ++j) {
      while (j < n && trainers[j] < players[i]) {
        ++j;
      }
      if (j == n) {
        return i;
      }
    }
    return m;
  }
};
```

23 LC 948 | Bag of Tokens (Medium)
Problem: A token can be played once: face up spends its value in power and gains one score; face down requires at least one score, spends one score, and gains its value in power. Choose any play order; maximize score from initial power and zero score.
Constraints: 0 <= tokens.length <= 1000; 0 <= tokens[i], power < 10^4

```
class Solution {
public:
  int bagOfTokensScore(vector<int>& tokens, int
    power) {
    ranges::sort(tokens);
    int ans = 0, score = 0;
    for (int i = 0, j = tokens.size() - 1; i <=
      j;) {
      if (power >= tokens[i]) {
        power -= tokens[i++];
        ans = max(ans, ++score);
      } else if (score > 0) {
        power += tokens[j--];
        --score;
      } else {
        break;
      }
    }
    return ans;
  }
};
```

30 LC 402 | Remove K Digits (Medium)
Problem: Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.
Constraints: 1 <= k <= num.length <= 10^5; num consists of only digits.; num does not have any leading zeros except for the zero itself.

```
class Solution {
public:
  string removeKdigits(string num, int k) {
    string stk;
    for (char& c : num) {
      while (k && stk.size() && stk.back() > c) {
        stk.pop_back();
        --k;
      }
      stk += c;
    }
    while (k--) {
      stk.pop_back();
    }
    int i = 0;
    for (; i < stk.size() && stk[i] == '0'; ++i) {
    }
    string ans = stk.substr(i);
    return ans == "" ? "0" : ans;
  }
};
```

34 LC 1877 | Minimize Maximum Pair Sum in Array (Medium)
Problem: Partition nums into pairs, using every element once. Minimize the largest pair sum and return that minimum.
Constraints: n == nums.length; 2 <= n <= 10^5; n is even.; 1 <= nums[i] <= 10^5

```
class Solution {
public:
  int minPairSum(vector<int>& nums) {
    ranges::sort(nums);
    int ans = 0, n = nums.size();
    for (int i = 0; i < n >> 1; ++i) {
      ans = max(ans, nums[i] + nums[n - i - 1]);
    }
    return ans;
  }
};
```

24 LC 846 | Hand of Straights (Medium)
Problem: Alice has some number of cards and she wants to rearrange the cards into groups so that each group is of size groupSize, and consists of groupSize consecutive cards. Given an integer array hand where hand[i] is the value written on the ith card and an integer groupSize, return true if she can rearrange the cards, or false otherwise.
Constraints: 1 <= hand.length <= 10^4; 0 <= hand[i] <= 10^9; 1 <= groupSize <= hand.length

```
class Solution {
public:
  bool isNStraightHand(vector<int>& hand, int
    groupSize) {
    if (hand.size() % groupSize) {
      return false;
    }
    ranges::sort(hand);
    unordered_map<int, int> cnt;
    for (int x : hand) {
      ++cnt[x];
    }
    for (int x : hand) {
      if (cnt.contains(x)) {
        for (int y = x; y < x + groupSize; ++y) {
          if (!cnt.contains(y)) {
            return false;
          }
          if (--cnt[y] == 0) {
            cnt.erase(y);
          }
        }
      }
    }
    return true;
  }
};
```

29 LC 1249 l Minimum Remove to Make Valid Parentheses (Medium)
Problem: Remove the fewest parentheses from s so its parentheses are balanced; keep all other characters and preserve order. Return any valid result.
Constraints: 1 <= s.length <= 10^5; s[i] is either '(' , ')', or lowercase English letter.

```
class Solution {
public:
  string minRemoveToMakeValid(string s) {
    string stk;
    int x = 0;
    for (char& c : s) {
      if (c == ')' && x == 0) continue;
      if (c == '(')
        ++x;
      else if (c == ')')
        --x;
      stk.push_back(c);
    }
    string ans;
    x = 0;
    while (stk.size()) {
      char c = stk.back();
      stk.pop_back();
      if (c == '(' && x == 0) continue;
      if (c == ')')
        ++x;
      else if (c == '(')
        --x;
      ans.push_back(c);
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

27 LC 678 | Valid Parenthesis String (Medium)

Problem: Return whether s can be balanced by interpreting each '*' independently as '(', ')', or an empty string. All fixed parentheses must match in order.

Constraints: 1 <= s.length <= 100; s[i] is '(', ')' or '*'.

```
class Solution {
public:
  bool checkValidString(string s) {
    int x = 0, n = s.size();
    for (int i = 0; i < n; ++i) {
      if (s[i] != ')') {
        ++x;
      } else if (x) {
        --x;
      } else {
        return false;
      }
    }
    x = 0;
    for (int i = n - 1; i >= 0; --i) {
      if (s[i] != '(') {
        ++x;
      } else if (x) {
        --x;
      } else {
        return false;
      }
    }
    return true;
  }
};
```

50 LC 1642 | Furthest Building You Can Reach (Medium)

Problem: Move through buildings in order. Descents cost nothing; an ascent costs its height difference in bricks or one ladder. Given bricks and ladders, return the furthest reachable zero-based building index.

Constraints: 1 <= heights.length <= 10^5; 1 <= heights[i] <= 10^6; 0 <= bricks <= 10^9; 0 <= ladders <= heights.length

```
class Solution {
public:
  int furthestBuilding(vector<int>& heights, int
    bricks, int ladders) {
    priority_queue<int, vector<int>, greater<int>>
      q;
    int n = heights.size();
    for (int i = 0; i < n - 1; ++i) {
      int a = heights[i], b = heights[i + 1];
      int d = b - a;
      if (d > 0) {
        q.push(d);
        if (q.size() > ladders) {
          bricks -= q.top();
          q.pop();
          if (bricks < 0) {
            return i;
          }
        }
      }
    }
    return n - 1;
  }
};
```

31 LC 316 | Remove Duplicate Letters (Hard)

Problem: Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Constraints: 1 <= s.length <= 10^4; s consists of lowercase English letters.

```
class Solution {
public:
  string removeDuplicateLetters(string s) {
    int n = s.size();
    int last[26] = {0};
    for (int i = 0; i < n; ++i) {
      last[s[i] - 'a'] = i;
    }
    string ans;
    int mask = 0;
    for (int i = 0; i < n; ++i) {
      char c = s[i];
      if ((mask >> (c - 'a')) & 1) {
        continue;
      }
      while (!ans.empty() && ans.back() > c &&
        last[ans.back() - 'a'] > i) {
        mask ^= 1 << (ans.back() - 'a');
        ans.pop_back();
      }
      ans.push_back(c);
      mask |= 1 << (c - 'a');
    }
    return ans;
  }
};
```

49 LC 630 | Course Schedule III (Hard)

Problem: There are n different online courses numbered from 1 to n. You are given an array courses where courses[i] = [duration_i, lastDay_i] indicate that the ith course should be taken continuously for duration_i days and must be finished before or on lastDay_i. You will start on the 1st day and you cannot take two or more courses simultaneously. Return the maximum number of courses that you can take.

Constraints: 1 <= courses.length <= 10^4; 1 <= duration_i, lastDay_i <= 10^4

```
class Solution {
public:
  int scheduleCourse(vector<vector<int>>& courses)
    {
    ranges::sort(courses, [](const vector<int>& a,
      const vector<int>& b) {
      return a[1] < b[1];
    });
    priority_queue<int> pq;
    int s = 0;
    for (auto& e : courses) {
      int duration = e[0], last = e[1];
      pq.push(duration);
      s += duration;
      while (s > last) {
        s -= pq.top();
        pq.pop();
      }
    }
    return pq.size();
  }
};
```

46 LC 2616 l Minimize the Maximum Difference of Pairs (Medium)

Problem: Choose p disjoint index pairs to minimize the largest absolute difference within any chosen pair. Return that minimum, or 0 if p=0.

Constraints: 1 <= nums.length <= 10^5; 0 <= nums[i] <= 10^9; 0 <= p <= (nums.length)/2

```
class Solution {
public:
  int minimizeMax(vector<int>& nums, int p) {
    ranges::sort(nums);
    int n = nums.size();
    int l = 0, r = nums[n - 1] - nums[0] + 1;
    auto check = [&](int diff) -> bool {
      int cnt = 0;
      for (int i = 0; i < n - 1; ++i) {
        if (nums[i + 1] - nums[i] <= diff) {
          ++cnt;
          ++i;
        }
      }
      return cnt >= p;
    };
    while (l < r) {
      int mid = (l + r) >> 1;
      if (check(mid)) {
        r = mid;
      } else {
        l = mid + 1;
      }
    }
    return l;
  }
};
```

48 LC 502 | IPO (Hard)

Problem: Choose at most k distinct projects. A project requires at least capital[i] current money and, when completed, adds profits[i] to it; the requirement is not spent. Start with w and maximize final capital.

Constraints: 1 <= k <= 10^5; 0 <= w <= 10^9; n == profits.length; n == capital.length; 1 <= n <= 10^5; 0 <= profits[i] <= 10^4; 0 <= capital[i] <= 10^9

```
using pii = pair<int, int>;
class Solution {
public:
  int findMaximizedCapital(int k, int w,
    vector<int>& profits, vector<int>& capital) {
    priority_queue<pii, vector<pii>, greater<pii>>
      q1;
    int n = profits.size();
    for (int i = 0; i < n; ++i) {
      q1.push({capital[i], profits[i]});
    }
    priority_queue<int> q2;
    while (k--) {
      while (!q1.empty() && q1.top().first <= w) {
        q2.push(q1.top().second);
        q1.pop();
      }
      if (q2.empty()) {
        break;
      }
      w += q2.top();
      q2.pop();
    }
    return w;
  }
};
```